

SS 1A

```

vboxuser@Ubuntu: ~/PES1UG24CS507-pes-vcs
make test_objects
./test_objects
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:100:47: warning: '/tmpXXXXXX' directive output may be truncated writing 10 bytes into a region of size between
1 and 512 [-Wformat-truncation=]
100 |     snprintf(temp_path, sizeof(temp_path), "%s/tmpXXXXXX", dir);
    |     ~~~~~^~~~~~
In file included from /usr/include/stdio.h:980,
from pes.h:8,
from object.c:3:
In function 'snprintf',
inlined from 'object_write' at object.c:100:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin__snprintf_chk' output between 11 and 522 bytes into
a destination of size 512
54 |     return __builtin__snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
    |            ^~~~~~
55 |                                     __glibc_objsize (__s), __fmt,
    |                                     ~~~~~
56 |                                     __va_arg_pack ());
    |                                     ~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.

```

SS 1B

```

vboxuser@Ubuntu: ~/PES1UG24CS507-pes-vcs
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:100:47: warning: '/tmpXXXXXX' directive output may be truncated writing 10 bytes into a region of size between
1 and 512 [-Wformat-truncation=]
100 |     snprintf(temp_path, sizeof(temp_path), "%s/tmpXXXXXX", dir);
    |     ~~~~~^~~~~~
In file included from /usr/include/stdio.h:980,
from pes.h:8,
from object.c:3:
In function 'snprintf',
inlined from 'object_write' at object.c:100:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin__snprintf_chk' output between 11 and 522 bytes into
a destination of size 512
54 |     return __builtin__snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
    |            ^~~~~~
55 |                                     __glibc_objsize (__s), __fmt,
    |                                     ~~~~~
56 |                                     __va_arg_pack ());
    |                                     ~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
vboxuser@Ubuntu: ~/PES1UG24CS507-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@Ubuntu: ~/PES1UG24CS507-pes-vcs$

```

SS 2A

```

56 |                                     __va_arg_pack ());
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function 'tree_from_index':
tree.c:135:13: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
135 |             object_write(OBJ_BLOB, buf, size, &e->hash);
    |             ^~~~~~
tree.c:139:13: warning: implicit declaration of function 'chdir' [-Wimplicit-function-declaration]
139 |             chdir(name);
    |             ^~~~~
tree.c:132:13: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
132 |             fread(buf, 1, size, f);
    |             ^~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
yboxuser@Ubuntu:~/PES1UG24CS507-pes-vcs$

```

SS 2B

```

00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050 blob 16.Hello, P
00000010: 4553 2d56 4353 210a ES-VCS!.

```

SS 3A

Staged changes:

staged: file1.txt

staged: file2.txt

Unstaged changes:

(nothing to show)

Untracked files:

untracked: tree.c

untracked: test_sequence.sh

untracked: pes.c

untracked: commit.c

untracked: test_objects.c

untracked: test_tree

untracked: index.c

untracked: test_tree.c

untracked: index.h

untracked: tree.h

untracked: README.md

untracked: commit.h

untracked: object.c

untracked: Makefile

untracked: pes.h

untracked: test_objects

SS 3B

```
100664 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776424638 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776424638 6 file2.txt
```

SS 4A

```
commit 5dfb36ccda68e3cc58a8660d3b763ae1a9a6234b33ad44c70295ce93c46f7589
Author: PES User <pes@localhost>
Date: 1776426791
```

Add bye file

```
commit a522ca894927e0196fb042291093a880e2d8f47708fd156a67ae240698157ffe
Author: PES User <pes@localhost>
Date: 1776426763
```

Add worldile

```
commit e9b16c9c971f1492ebf4f0e94e6987e85d2d6dfc3f744312e55a72fbf3cb196d
Author: PES User <pes@localhost>
Date: 1776426686
```

Initial committial commiQ

SS 4B

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/48/1407f7237f7bd749dca31e2d8422baf211741d5556f007e0faf821ffc7179
.pes/objects/5d/fb36ccda68e3cc58a8660d3b763ae1a9a6234b33ad44c70295ce93c46f7589
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7d/c3b9d4cee22f645e501f8aaa822f6777b3198434adc74491632478799e9e8b
.pes/objects/92/19086282c09f0db9088484e1ef89f1b05ecfb420a8784a6de327022771fbec
.pes/objects/a5/22ca894927e0196fb042291093a880e2d8f47708fd156a67ae240698157ffe
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e9/b16c9c971f1492ebf4f0e94e6987e85d2d6dfc3f744312e55a72fbf3cb196d
.pes/objects/fe/bed41618fe89b49ea9b82684254a1e74aa7f80fdb38fc84b592ccca955267
.pes/refs/heads/main
```

Q5.1: How would you implement pes checkout <branch>? What makes it complex?

A branch is a file in `.pes/refs/heads/` containing a commit hash. To implement `pes checkout <branch>`:

1. **Check if the branch exists**
Verify that `.pes/refs/heads/<branch>` exists. If not, return an error.
2. **Update HEAD**
Write:

`ref: refs/heads/<branch>`

into `.pes/HEAD`.
3. **Read commit hash**
Read the commit hash stored in `.pes/refs/heads/<branch>`.
4. **Load commit and tree**
Read the commit object, extract the root tree hash, and load the tree.
5. **Update working directory**
 - Remove tracked files from the current working directory
 - Recreate files and directories from the target tree
 - Write blob contents into files

Why it is complex:

- Must avoid overwriting uncommitted changes
 - Requires recursive traversal of trees
 - Needs careful file deletion and recreation
 - Must maintain consistency between index, working directory, and object store
-

Q5.2: How to detect dirty working directory conflicts?

To detect uncommitted changes:

1. For each file in the index:
 - Compare stored metadata (mtime, size) with the working directory
 - If different → recompute hash and compare with stored hash
2. A file is **modified (dirty)** if:
 - It is missing
 - Its metadata differs and hash differs
3. Conflict occurs if:
 - Working directory version $\neq$ index version
 - AND index version $\neq$ target branch version

If both conditions hold, checkout should be aborted to prevent data loss.

Q5.3: What is detached HEAD and how to recover?

Detached HEAD occurs when `.pes/HEAD` contains a commit hash instead of a branch reference.

In this state:

- New commits can still be created
- But no branch points to them
- These commits become unreachable (dangling)

Recovery:

- Create a new branch pointing to the current commit
 - Or manually use the commit hash from logs to reattach
-

Q6.1: How to find and delete unreachable objects?

Algorithm:

1. Start from all branch heads (`.pes/refs/heads/*`)
2. Traverse all reachable commits:
 - Mark commit
 - Follow parent links
 - Follow tree references
3. Traverse trees:
 - Mark tree
 - Follow all blobs and subtrees
4. Maintain a **set of reachable hashes** (using a hash set)
5. Iterate through `.pes/objects/`:
 - Delete objects not present in the reachable set

Data structure:

- Hash set (for efficient lookup of visited objects)

Estimate:

- For 100,000 commits and 50 branches
 - Total objects visited $\approx$ 1M–10M (including trees and blobs)
-

Q6.2: Why is garbage collection dangerous during commits?

Race condition:

1. Garbage collection starts and scans reachable objects
 2. A commit is being created simultaneously:
 - New blobs and trees are written
 - Commit not yet referenced by any branch
 3. GC does not see these objects as reachable
 4. GC deletes them
 5. Commit finishes and references deleted objects $\rightarrow$ repository corruption
-

How Git avoids this:

- Writes objects before updating references (atomic ordering)
- Uses locking to prevent concurrent GC and commit conflicts
- Keeps recently created objects temporarily (grace period)
- Uses packfiles and internal tracking for safe cleanup